

未命名文档,

BalanceImp

一个问题是 `addBalPairs.insertTree(address, targetPair);` 会插入null到tree

一个问题是 `totalSupply` 没有更新 导致repok会false

java

```
1 @Override
2     public void updateBalance(String address, int newBalance) {
3         AddressBalancePair targetPair = addBalPairs.searchTree(address);
4         // O(log |A|)
5         //!!!!!!!!!!!!!!!!!!!!
6         int oldBalance = (targetPair == null) ? 0 :
7 targetPair.balance; //fix totalSupply doesn't update
8         if ( address.equals("0") && allAddress.length() == 0 ) {
9             // initiat `totalSupply` with the amount of BB-coins of
10             // the address "0" in the genesis block
11             ////!!!bug!!! addBalPairs.insertTree(address, targetPair);
12             addBalPairs.insertTree(address, new AddressBalancePair(address,
13 newBalance));
14             allAddress.insertRear(address);
15         // O(log |A|)
16             //fix using above 2 line
17             totalSupply = newBalance;
18         } else if ( targetPair == null ) {
19             // address does not exist in Balance
20             addBalPairs.insertTree(address, new AddressBalancePair(address,
21 newBalance)); // O(log |A|)
22             allAddress.insertRear(address);
23         // O(1)
24             totalSupply += newBalance; //!!!!!!!!!!!!!!!!!!!!
25         } else {
26             // address already exists in Balance
27             addBalPairs.updateTree(address, new AddressBalancePair(address,
28 newBalance));
29             totalSupply += (newBalance - oldBalance); //!!!!!!!!!!!!!!!!!!!!
30         }
31         if ( !repOK() ) {
32             throw new IllegalStateException(
33                 "BalanceImp.updateBalance(): totalSupply does not equal to
34 the sum of all accounts."
35             );
36         }
37     }
38 }
```

```
}  
}
```

AVLTreeImp

parentChildRelationship() 这个j是不是要重新初始化 每一次i变的时候

java

```
1 private boolean parentChildRelationship() {  
2     boolean checkeFlag = true;  
3     Sequence<AVLNode<K, V>> nodes = nodesSequence();  
4     int i = 0;  
5     int j = 0;  
6     while ( checkeFlag && i < nodesNum ) {  
7         while ( checkeFlag && j < nodesNum ) {  
8             AVLNode<K, V> node1 = nodes.at(i);  
9             AVLNode<K, V> node2 = nodes.at(j);  
10            if (  
11                node1.left == node2 && node2.parent != node1 ||  
12                node1.right == node2 && node2.parent != node1  
13            ) checkeFlag = false;  
14            else j++;  
15        }  
16        i++;  
17    }  
18    return checkeFlag;  
19 }
```

ListoverLinkedList

AI说这个 insertSorted is not generic-safe and can NPE at the tail.

java

```
1 public void insertSorted(T elem) {  
2     Node<T> newNode = new Node<T>(elem);  
3     if (head == null) {  
4         head = newNode;  
5     } else {  
6         Node<T> prev = findPrev(elem);  
7         if (prev == null) {  
8             newNode.next = head;  
9             head.prev = newNode;  
10            head = newNode;  
11        }  
12    }  
13 }
```

```

11         } else {
12             newNode.prev = prev;
13             newNode.next = prev.next;
14             prev.next.prev = newNode;
15             prev.next = newNode;
16         }
17     }
18     length++;
19     if ( !repOk() )
20         throw new IllegalStateException(
21             "RI of ListoverLinkedList is not satisfied."
22         );
23 }

```

to

```

java
1  @Override
2  public void insertSorted(T elem) {
3      Node<T> newNode = new Node<T>(elem);
4      if (head == null) {
5          head = newNode;
6          tail = newNode;
7      } else {
8          Node<T> prev = findPrev(elem);
9          if (prev == null) {
10             newNode.next = head;
11             head.prev = newNode;
12             head = newNode;
13         } else if (prev.next == null) {
14             // insert at tail
15             prev.next = newNode;
16             newNode.prev = prev;
17             tail = newNode;
18         } else {
19             newNode.prev = prev;
20             newNode.next = prev.next;
21             prev.next.prev = newNode;
22             prev.next = newNode;
23         }
24     }
25     length++;
26     if ( !repOk() )
27         throw new IllegalStateException(
28             "RI of ListoverLinkedList is not satisfied."

```

```
29         );
30     }
```

findPrev

java

```
1 private Node<T> findPrev(T elem) {
2     Node<T> curr = head;
3     Node<T> prev = null;
4     //!!!while (curr != null && (Integer) curr.data < (Integer) elem) {
5     while (curr != null && ((Comparable<T>) curr.data).compareTo(elem) < 0)
6 {
7     prev = curr;
8     curr = curr.next;
9 }
10    return prev;
11 }
```

LinkedListQueue

When you **remove the last element** via `remove(size-1)`, you don't update `tail`, so `tail` still points to the removed node.

java

```
1 @Override
2 public void remove(int index) {
3     // Remove element at index while maintaining order
4     if (index < 0 || index >= size) {
5         throw new IndexOutOfBoundsException("Index out of bounds");
6     }
7     if (index == 0) {
8         dequeue();
9         return;
10    }
11    Node curr = head;
12    for (int i = 0; i < index - 1; i++) {
13        curr = curr.next;
14    }
15    //!!!curr.next = curr.next.next;
16    //add!!!
17    Node toRemove = curr.next;
18    curr.next = toRemove.next;
19    if (toRemove == tail) {
20        tail = curr;
21    }
```

```
22      //see above
23      size--;
24  }
```

Similarly, `dequeue()` when size becomes 0 doesn't reset `tail`.

```
java
1      public TransactionWithFee dequeue() {
2          if (head == null) {
3              throw new RuntimeException("Queue is empty");
4          }
5          TransactionWithFee elem = head.data;
6          head = head.next;
7          size--;
8          //!!!!
9          if (head == null) {
10             tail = null;
11         }
12         //see above!!!
13         return elem;
14     }
```